

C++ Codebook

This codebook contains commonly used algorithms and utilities optimized for competitive programming.

1. Basic Utilities

IO Speedup (cin, cout)

```
1 cin.sync_with_stdio(0);
2 cin.tie(0);
```

Buffered Output (Defer Flushing Until Program End)

```
1 ios::sync_with_stdio(false);
2 cin.tie(NULL);
```

Test With File

```
1 g++ main.cpp -o main
2 main < input.in > output.out
```

Write File

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 int main() {
6     std::ofstream outFile("example.txt");
7     if (!outFile.is_open()) {
8         std::cerr << "Error opening file for writing!"
9         << std::endl;
10        return 1;
11    }
12    outFile << "Hello, World!\n";
13    outFile << "This is a line in a text file.\n";
14    outFile << 123 << " " << 45.67 << "\n";
15    outFile.close();
16    return 0;
17 }
```

Read File

```
1 #include <iostream>
2 #include <fstream>
3 #include <string>
4
5 int main() {
6     std::ifstream inFile("example.txt");
7     if (!inFile.is_open()) {
8         std::cerr << "Error opening file for reading!"
9         << std::endl;
10        return 1;
11    }
12    std::string line;
13    while (std::getline(inFile, line)) {
14        std::cout << line << std::endl;
15    }
16    inFile.close();
17    return 0;
18 }
```

Greatest Common Divisor (GCD)

```
1 int GCD(int x, int y) {
2     if (!x) return y;
3     if (!y) return x;
4     while ((x %= y) && (y %= x));
5     return x + y;
6 }
```

Least Common Multiple (LCM)

```
1 int LCM(int num1, int num2) {
2     return ((num1 * num2) / GCD(num1, num2));
3 }
```

GCD and LCM in C++17

```
1 #include <numeric>
2 int main () {
3     int a = 48, b = 18;
4     int gcd = std::gcd(a, b);
5     int lcm = std::lcm(a, b);
6     return 0;
7 }
```

Priority Queue (std::priority_queue)

Description: Demonstrates the use of C++ built-in `std::priority_queue` (from `<queue>`), including default behavior (Max-Heap), built-in Min-Heap, and Custom Struct Sorting. - **Default:** Max-Heap (largest element at the top). - **Built-in Min-Heap:** Uses `std::greater` (smallest element at the top). Very common with `std::pair` in graph algorithms. - **Custom Struct Sorting:** Overload the `<` operator inside a custom struct.

```
1 #include <queue>
2 #include <vector>
3 #include <iostream>
4 #include <utility> // for pair
5
6 using namespace std;
7
8 // Struct with custom sorting rules
9 struct Item {
10     int id;
11     int priority;
12
13     // Custom sorting rule: sort by priority ascending
14     < (min-heap)
15     // If priority is equal, sort by id ascending
16     < (smaller id at the top)
17     // Note: Since priority_queue is a Max-Heap by
18     < default, we invert the comparison logic.
19     bool operator<(const Item& other) const {
20         if (priority != other.priority) {
21             return priority > other.priority; // Larger
22             < priority means "less" priority in
23             < max-heap (so smaller comes to top)
24         }
25     }
26 }
```

```

20     return id > other.id; // Larger ID means "less"
    ⇨ priority (so smaller ID comes to top)
21 }
22 };
23
24 void priorityQueueUsage() {
25     priority_queue<int> max_pq;
26     max_pq.push(10);
27     max_pq.push(30);
28     max_pq.push(20);
29     // max_pq.top() is 30 (largest element first)
30
31     priority_queue<int, vector<int>, greater<int>>
    ⇨ min_pq;
32     min_pq.push(10);
33     min_pq.push(30);
34     min_pq.push(20);
35     // min_pq.top() is 10 (smallest element first)
36
37     priority_queue<pair<int, int>, vector<pair<int,
    ⇨ int>>, greater<pair<int, int>>> pair_pq;
38     pair_pq.push({5, 101}); // {distance, node}
39     pair_pq.push({2, 102});
40     pair_pq.push({5, 100});
41     // pair_pq.top() is {2, 102} (smallest distance
    ⇨ first, then smallest node index)
42
43     priority_queue<Item> custom_pq;
44     custom_pq.push({101, 5});
45     custom_pq.push({102, 5});
46     custom_pq.push({103, 2});
47     // custom_pq.top() is {103, 2} (priority 2), then
    ⇨ {101, 5} (priority 5, smaller id first)
48 }

```

2. Prime Numbers

Count Primes (Sieve of Eratosthenes)

Time Complexity: $O(N \log \log N)$

Space Complexity: $O(N)$ **Usage:** Efficiently counts large ranges of primes.

```

1  #include <vector>
2  #include <algorithm>
3
4  using namespace std;
5
6  int countPrimes(int N) {
7      if (N < 2) return 0;
8      vector<bool> isPrime(N + 1, true);
9      isPrime[0] = isPrime[1] = false;
10     for (int i = 2; i * i <= N; ++i) {
11         if (isPrime[i]) {
12             for (int j = i * i; j <= N; j += i) {
13                 isPrime[j] = false;
14             }
15         }
16     }
17     return count(isPrime.begin(), isPrime.end(), true);
18 }

```

Prime Number Test

Description: Tests if a given 64-bit integer n is prime using $O(\sqrt{n})$ trial division optimized with a $6k \pm 1$ rule.

Time Complexity: $O(\sqrt{n})$

Space Complexity: $O(1)$ **Usage:** Fast primality test for single large integers.

```

1  bool isPrime(long long n) {
2      if (n <= 1) return false;
3      if (n <= 3) return true;
4      if (n % 2 == 0 || n % 3 == 0) return false;
5      for (long long i = 5; i * i <= n; i += 6) {
6          if (n % i == 0 || n % (i + 2) == 0) {
7              return false;
8          }
9      }
10     return true;
11 }

```

List Primes (Sieve of Eratosthenes)

Description: Generates and returns a list of all prime numbers less than or equal to N . **Time Complexity:** $O(N \log \log N)$

Space Complexity: $O(N)$

```

1  #include <vector>
2
3  using namespace std;
4
5  vector<int> sieve(int N) {
6      vector<bool> isPrime(N + 1, true);
7      isPrime[0] = isPrime[1] = false;
8      for (int i = 2; i * i <= N; ++i) {
9          if (isPrime[i]) {
10             for (int j = i * i; j <= N; j += i) {
11                 isPrime[j] = false;
12             }
13         }
14     }
15     vector<int> primes;
16     for (int i = 2; i <= N; ++i) {
17         if (isPrime[i]) {
18             primes.push_back(i);
19         }
20     }
21     return primes;
22 }

```

Prime Factorization

Description: Performs prime factorization on a 64-bit integer n . **Time Complexity:** $O(\sqrt{n})$ **Space Complexity:** $O(\log n)$ (number of unique prime factors) **Returns:** A list of pairs where each pair is {base, exponent} (representing the base prime and its exponent/power).

```

1  #include <vector>
2  #include <utility>
3
4  using namespace std;
5

```

```

6 // Returns a vector of pairs: {prime_base, exponent}
7 vector<pair<long long, int>> primeFactorization(long
  ↳ long n) {
8     vector<pair<long long, int>> factors;
9     for (long long i = 2; i * i <= n; ++i) {
10         if (n % i == 0) {
11             int count = 0;
12             while (n % i == 0) {
13                 count++;
14                 n /= i; // Eliminate the factor
15             }
16             factors.push_back({i, count});
17         }
18     }
19     if (n > 1) {
20         factors.push_back({n, 1}); // Remaining prime
  ↳ factor
21     }
22     return factors;
23 }

```

3. String Processing

String Split (Regex)

Description: Splits a string into substrings based on a regular expression pattern delimiter. **Parameters:** - **str:** The target string to split. - **keys:** Regex pattern specifying the delimiters (default matches dashes - or commas ,). - **keep:** If **true**, retains the matched delimiters as separate tokens in the output. **Usage:** Ideal for parsing complex string inputs. Requires `<regex>`.

```

1 #include <vector>
2 #include <string>
3 #include <regex>
4
5 using namespace std;
6
7 vector<string> split(const string& str, string keys =
  ↳ "{-1,3}|,)", bool keep = false) {
8     vector<string> tokens;
9     regex re(keys);
10    sregex_token_iterator it;
11    if (keep) {
12        it = sregex_token_iterator(str.begin(),
  ↳ str.end(), re, {-1, 0});
13    } else {
14        it = sregex_token_iterator(str.begin(),
  ↳ str.end(), re, -1);
15    }
16    sregex_token_iterator end;
17    while (it != end) {
18        tokens.push_back(*it);
19        ++it;
20    }
21    return tokens;
22 }

```

Knuth-Morris-Pratt (KMP) Substring Search

Description: High-performance pattern searching algorithm. Precomputes the Longest Prefix Suffix (LPS)

table to skip redundant character comparisons, eliminating the need to backtrack the text index. **Time Complexity:** $O(N + M)$ where N is text length, M is pattern length. **Space Complexity:** $O(M)$ for the LPS table. **Usage:** Fast substring matching when brute-force $O(N \cdot M)$ is too slow.

```

1 #include <vector>
2 #include <string>
3
4 using namespace std;
5
6 // Build the Longest Prefix which is also a Suffix
  ↳ (LPS) table
7 vector<int> buildLPS(const string &pattern) {
8     int m = pattern.size();
9     vector<int> lps(m, 0);
10    int len = 0; // Length of the previous longest
  ↳ prefix suffix
11    for (int i = 1; i < m; ) {
12        if (pattern[i] == pattern[len]) {
13            len++;
14            lps[i] = len;
15            i++;
16        } else {
17            if (len != 0) {
18                len = lps[len - 1]; // Backtrack in
  ↳ pattern
19            } else {
20                lps[i++] = 0;
21            }
22        }
23    }
24    return lps;
25 }
26
27 // KMP search main function: Returns starting indices
  ↳ of matches
28 vector<int> KMPsearch(const string &text, const string
  ↳ &pattern) {
29     vector<int> result;
30     int n = text.size(), m = pattern.size();
31     if (m == 0) return result;
32     vector<int> lps = buildLPS(pattern);
33     int i = 0, j = 0; // i -> text index, j -> pattern
  ↳ index
34     while (i < n) {
35         if (text[i] == pattern[j]) {
36             i++; j++;
37             if (j == m) {
38                 result.push_back(i - j); // Found match
  ↳ at index (i - j)
39                 j = lps[j - 1];
40             }
41         } else {
42             if (j != 0) {
43                 j = lps[j - 1];
44             } else {
45                 i++;
46             }
47         }
48     }
49     return result;
50 }

```

4. Computational Geometry

Closest Pair of Points (Divide & Conquer)

Description: Finds the closest pair of points in a 2D Euclidean plane using a divide-and-conquer strategy.

1. **Sort:** Sort all points by X-coordinates. 2. **Divide & Solve:** Recursively find the minimum distance in left and right halves. 3. **Combine:** Check points within a vertical strip of width $2d$ around the midline. Sort them by Y-coordinates and compare each point with at most 6 subsequent points. **Time Complexity:** $O(N \log N)$

Space Complexity: $O(N)$ **Usage:** Standard high-performance solution for closest-pair problems.

```

1  #include <vector>
2  #include <cmath>
3  #include <algorithm>
4  #include <limits>
5  #include <utility>
6
7  using namespace std;
8
9  struct Point {
10     double x, y;
11 };
12
13 // Calculate Euclidean distance between two points
14 double dist(const Point& a, const Point& b) {
15     return hypot(a.x - b.x, a.y - b.y);
16 }
17
18 // Brute force method for 3 or fewer points
19 double bruteForce(vector<Point>& points, int left, int
20     ↪ right, pair<Point, Point>& bestPair) {
21     double minDist = numeric_limits<double>::max();
22     for (int i = left; i <= right; ++i) {
23         for (int j = i + 1; j <= right; ++j) {
24             double d = dist(points[i], points[j]);
25             if (d < minDist) {
26                 minDist = d;
27                 bestPair = {points[i], points[j]};
28             }
29         }
30     }
31     return minDist;
32 }
33
34 // Check the strip area for closer points crossing the
35     ↪ divide boundary
36 double stripClosest(vector<Point>& strip, double d,
37     ↪ pair<Point, Point>& bestPair) {
38     double minDist = d;
39     sort(strip.begin(), strip.end(), [](const Point& a,
40     ↪ const Point& b) {
41         return a.y < b.y;
42     });
43     for (size_t i = 0; i < strip.size(); ++i) {
44         // Compare each point with next points inside
45         ↪ the strip
46         for (size_t j = i + 1; j < strip.size() &&
47         ↪ (strip[j].y - strip[i].y) < minDist; ++j) {
48             double newDist = dist(strip[i], strip[j]);
49             if (newDist < minDist) {
50                 minDist = newDist;

```

```

45         bestPair = {strip[i], strip[j]};
46     }
47 }
48 }
49 return minDist;
50 }
51
52 // Recursive Divide & Conquer helper utility
53 double closestUtil(vector<Point>& pointsSortedByX, int
54     ↪ left, int right, pair<Point, Point>& bestPair) {
55     // Use brute force if points count is 3 or less
56     if (right - left <= 3) {
57         return bruteForce(pointsSortedByX, left,
58         ↪ right, bestPair);
59     }
60     int mid = (left + right) / 2;
61     Point midPoint = pointsSortedByX[mid];
62     pair<Point, Point> leftPair, rightPair;
63     double dl = closestUtil(pointsSortedByX, left,
64     ↪ mid, leftPair);
65     double dr = closestUtil(pointsSortedByX, mid + 1,
66     ↪ right, rightPair);
67     double d = dl;
68     bestPair = leftPair;
69     if (dr < dl) {
70         d = dr;
71         bestPair = rightPair;
72     }
73     // Gather points inside vertical strip of width 2d
74     vector<Point> strip;
75     for (int i = left; i <= right; ++i) {
76         if (abs(pointsSortedByX[i].x - midPoint.x) < d)
77         ↪ {
78             strip.push_back(pointsSortedByX[i]);
79         }
80     }
81     pair<Point, Point> stripPair;
82     double dStrip = stripClosest(strip, d, stripPair);
83     if (dStrip < d) {
84         d = dStrip;
85         bestPair = stripPair;
86     }
87     return d;
88 }
89
90 // Main entry function to find the closest pair of
91     ↪ points
92 pair<Point, Point> findClosestPair(vector<Point>&
93     ↪ points) {
94     vector<Point> pointsSortedByX = points;
95     sort(pointsSortedByX.begin(),
96     ↪ pointsSortedByX.end(), [](const Point& a, const
97     ↪ Point& b) {
98         return a.x < b.x;
99     });
100     pair<Point, Point> bestPair;
101     closestUtil(pointsSortedByX, 0,
102     ↪ pointsSortedByX.size() - 1, bestPair);
103     return bestPair;
104 }

```

5. Permutation Operations

Next Permutation (Array-based)

Description: Rearranges the array elements into the lexicographically next greater permutation of elements. If no such permutation exists (i.e., the array is sorted in descending order), it rearranges it into the lowest possible order (sorted in ascending order).

Time Complexity: $O(N)$

Space Complexity: $O(1)$ auxiliary space **Usage:** Finds the next permutation of a vector/array in-place or returns a copy of the next permutation.

```
1  #include <vector>
2  #include <algorithm>
3
4  using namespace std;
5
6  // Returns the lexicographically next permutation of
7  // the array.
8  vector<int> nextPermutation(vector<int> nums) {
9      int n = nums.size();
10     int i = n - 2;
11     // Find the first decreasing element from the right
12     while (i >= 0 && nums[i] >= nums[i + 1]) {
13         i--;
14     }
15     if (i >= 0) {
16         // Find the successor to nums[i] from the right
17         int j = n - 1;
18         while (nums[j] <= nums[i]) {
19             j--;
20         }
21         swap(nums[i], nums[j]);
22     }
23     // Reverse the remaining ascending suffix
24     reverse(nums.begin() + i + 1, nums.end());
25     return nums;
26 }
```

Permutation Rank (Lexicographical Rank)

Description: Calculates the 1-based lexicographical rank (index) of a permutation. This implementation handles generic values by using coordinate compression, uses a Fenwick Tree (Binary Indexed Tree) for $O(N \log N)$ complexity, and supports modulo arithmetic to prevent integer overflow for larger arrays.

Time Complexity: $O(N \log N)$

Space Complexity: $O(N)$ **Usage:** Given a permutation of unique elements, returns its 1-based rank modulo $10^9 + 7$.

```
1  #include <vector>
2  #include <algorithm>
3
4  using namespace std;
5
6  // Binary Indexed Tree (Fenwick Tree) helper
7  struct FenwickTree {
8      int n;
9      vector<int> tree;
10     FenwickTree(int n) : n(n), tree(n + 1, 0) {}
11 }
```

```
11 void update(int i, int delta) {
12     for (; i <= n; i += i & -i) tree[i] += delta;
13 }
14 int query(int i) {
15     int sum = 0;
16     for (; i > 0; i -= i & -i) sum += tree[i];
17     return sum;
18 }
19 };
20
21 // Calculates the 1-based lexicographical rank of a
22 // permutation.
23 // Assumes unique elements. Returns rank modulo MOD
24 // (default is 1e9 + 7).
25 long long getPermutationRank(vector<int> nums, long
26 // long MOD = 1000000007) {
27     int n = nums.size();
28     if (n == 0) return 1;
29
30     // Precompute factorials modulo MOD
31     vector<long long> fact(n, 1);
32     for (int i = 1; i < n; ++i) {
33         fact[i] = (fact[i - 1] * i) % MOD;
34     }
35
36     // Coordinate compression to map values to [1, n]
37     vector<int> temp = nums;
38     sort(temp.begin(), temp.end());
39     vector<int> compressed(n);
40     for (int i = 0; i < n; ++i) {
41         compressed[i] = lower_bound(temp.begin(),
42 // temp.end(), nums[i]) - temp.begin() + 1;
43     }
44
45     FenwickTree bit(n);
46     for (int i = 1; i <= n; ++i) {
47         bit.update(i, 1);
48     }
49
50     long long rank = 1;
51     for (int i = 0; i < n; ++i) {
52         // Count how many unused elements to the right
53         // are smaller than compressed[i]
54         int smallerCount = bit.query(compressed[i] -
55 // 1);
56         rank = (rank + smallerCount * fact[n - 1 - i])
57 // % MOD;
58         // Mark current element as used
59         bit.update(compressed[i], -1);
60     }
61     return rank;
62 }
```

6. Math

Euler Totient Function

Description: Calculates Euler's totient function $\phi(x)$, which counts the number of positive integers strictly less than x that are coprime to x . **Time Complexity:** $O(\sqrt{x})$

Space Complexity: $O(1)$ **Usage:** Commonly used in number theory, particularly for finding modular multiplicative inverses and applying Euler's theorem.

```

1  #include <cmath>
2
3  int Phi(int x) {
4      if (x < 2) return 0;
5      int ret = x;
6      int sq = sqrt(x);
7      for (int p = 2; p <= sq; p++) {
8          if (x % p == 0) {
9              while (x % p == 0) x /= p;
10             ret -= ret / p;
11         }
12         if (x == 1) break;
13     }
14     if (x > 1) ret -= ret / x;
15     return ret;
16 }

```

7. Range Queries

Difference Array

Description: Efficiently applies multiple range addition updates $[L, R]$ by modifying only the boundaries of a difference array. A prefix sum is then used to reconstruct the final array values. **Returns:** The minimum coverage value across all positions after applying all updates. **Time Complexity:** $O(M + N)$ where M is the number of updates and N is the array size. **Space Complexity:** $O(N)$ for the difference array. **Usage:** Ideal for scenarios requiring many range updates followed by a single full-array query or state evaluation.

```

1  #include <iostream>
2  #include <vector>
3  #include <climits>
4
5  using namespace std;
6
7  long long differenceArray() {
8      ios::sync_with_stdio(false);
9      cin.tie(nullptr);
10
11     long long N, M;
12     if (!(cin >> N >> M)) return -1;
13
14     // Size N+2 to prevent R+1 from exceeding boundary
15     vector<long long> diff(N + 2, 0LL);
16
17     // Read interval [L, R] for each turret
18     for (int i = 0; i < M; i++) {
19         long long L, R;
20         cin >> L >> R;
21         // Difference array update: +1 at start L, -1
22         // at position after end R+1
23         diff[L] += 1;
24         if (R + 1 <= N) {
25             diff[R + 1] -= 1;
26         }
27     }
28
29     // Compute prefix-sum directly on diff array and
30     // track the minimum value

```

```

29     long long cover = 0;           // Current cumulative
30     // coverage at the current position
31     long long answer = LLONG_MAX; // Stores the minimum
32     // coverage value
33
34     for (int pos = 1; pos <= N; pos++) {
35         cover += diff[pos]; // diff[pos] indicates the
36         // change in coverage at pos
37         // cover represents the actual coverage count
38         // at pos
39         if (cover < answer) {
40             answer = cover;
41         }
42     }
43
44     // Under normal circumstances, answer is at least
45     // 0.
46     // Return the answer.
47     return answer;
48 }

```

Fenwick Tree (Binary Indexed Tree)

Description: Support Point Update, Prefix Sum Query, Range Sum Query **Complexity:** Update : $O(\log N)$, Query : $O(\log N)$, Memory : $O(N)$

```

1  struct BIT {
2      int n;
3      vector<long long> bit;
4
5      BIT(int _n) {
6          n = _n;
7          bit.assign(n + 1, 0);
8      }
9
10     void add(int idx, long long val) {
11         while (idx <= n) {
12             bit[idx] += val;
13             idx += idx & -idx;
14         }
15     }
16
17     long long sum(int idx) {
18         long long res = 0;
19
20         while (idx > 0) {
21             res += bit[idx];
22             idx -= idx & -idx;
23         }
24
25         return res;
26     }
27
28     long long query(int l, int r) {
29         return sum(r) - sum(l - 1);
30     }
31 };

```

Note: - `lowbit[i]` returns the size of the range managed by index `i`:

```
1 lowbit(x) = x & -x
```

- `tree[i]` stores the sum for the range managed by index

8. Graph Algorithms

Minimum Spanning Tree (Kruskal's Algorithm)

Description: Finds the Minimum Spanning Tree (MST) of a connected, undirected, weighted graph. It uses the greedy approach: sorts all edges in non-decreasing order of their weight, and adds edges one by one if they don't form a cycle. A Disjoint Set Union (DSU) structure is used to check and manage cycles efficiently. **Time Complexity:** $O(E \log E + E\alpha(V))$ where E is the number of edges, V is the number of vertices, and α is the Inverse Ackermann function. **Space Complexity:** $O(V)$ for the DSU parent and rank arrays. **Usage:** Suitable for sparse graphs. Supports 1-indexed vertices by default.

```
1  #include <vector>
2  #include <algorithm>
3
4  using namespace std;
5
6  // Representation of an edge in the graph
7  struct Edge {
8      int u, v;    // u, v: The two endpoint vertices
9      // (nodes) connected by this edge
10     long long w; // w: The weight (cost/distance) of
11     // this edge
12     // Overload the < operator to sort edges by weight
13     bool operator<(const Edge& other) const {
14         return w < other.w;
15     }
16 };
17
18 // Disjoint Set Union (DSU) / Union-Find structure
19 struct DSU {
20     vector<int> parent; // parent[i] stores the
21     // parent/leader of element i
22     vector<int> rank;   // rank[i] stores the
23     // approximate depth of the tree rooted at i (for
24     // Union by Rank optimization)
25
26     DSU(int n) { // n: The number of vertices/elements
27     // in the graph
28         parent.resize(n + 1);
29         rank.resize(n + 1, 0);
30         for (int i = 0; i <= n; i++) {
31             parent[i] = i; // Initially, every vertex
32             // is its own parent (disjoint set)
33         }
34
35         // Finds the representative (root leader) of the
36         // set containing element i
37         int find(int i) {
38             if (parent[i] == i)
39                 return i;
40             return parent[i] = find(parent[i]); // Path
41             // compression optimization
42         }
43
44         // Merges the set containing element i with the set
45         // containing element j.
```

```
37 // Returns true if they were in different sets and
38 // successfully merged; false if they were already
39 // in the same set.
40 bool unite(int i, int j) {
41     int root_i = find(i);
42     int root_j = find(j);
43     if (root_i != root_j) {
44         // Union by rank optimization: attach the
45         // smaller tree under the larger tree
46         if (rank[root_i] < rank[root_j]) {
47             parent[root_i] = root_j;
48         } else if (rank[root_i] > rank[root_j]) {
49             parent[root_j] = root_i;
50         } else {
51             parent[root_j] = root_i;
52             rank[root_i]++;
53         }
54         return true; // Successfully merged
55     }
56     return false; // Already in the same set
57     // (connecting them would form a cycle)
58 }
59
60 // Computes the MST weight and optionally stores the
61 // selected edges.
62 // n: Number of vertices (nodes) in the graph.
63 // edges: Input vector containing all edges in the
64 // graph (will be sorted in-place).
65 // mstEdges: Output vector to store the selected edges
66 // that form the MST.
67 // Returns the total MST weight, or -1 if the graph is
68 // disconnected (no MST exists).
69 long long kruskal(int n, vector<Edge>& edges,
70 // vector<Edge>& mstEdges) {
71     mstEdges.clear();
72     sort(edges.begin(), edges.end()); // Sort edges in
73     // ascending order of weight (greedy approach)
74
75     DSU dsu(n); // Disjoint Set Union to detect cycles
76     long long mst_weight = 0; // Accumulator for the
77     // total weight of the MST
78     int edges_used = 0; // Counter for the number of
79     // edges added to the MST
80
81     for (const auto& edge : edges) {
82         // Try to connect the two endpoints of the
83         // current edge (edge.u and edge.v)
84         if (dsu.unite(edge.u, edge.v)) {
85             mst_weight += edge.w;
86             mstEdges.push_back(edge);
87             edges_used++;
88             // A tree with n vertices always has
89             // exactly n - 1 edges
90             if (edges_used == n - 1) {
91                 break;
92             }
93         }
94     }
95
96     if (edges_used == n - 1) {
97         return mst_weight;
98     }
99     return -1; // Graph is not fully connected (could
100     // not select n - 1 edges)
101 }
```

Bipartite Matching (Hungarian Algorithm)

Finds the maximum matching in a bipartite graph. Nodes in set U are indexed 0 to $n - 1$. Nodes in set V are indexed 0 to $m - 1$.

```

1  #include <vector>
2
3  struct Hungarian {
4      int n, m;
5      std::vector<std::vector<int>> adj; // adj[u]
        ↳ contains neighbors in V
6      std::vector<int> match_v;          // match_v[v]
        ↳ stores the matched u node
7      std::vector<bool> visited;
8
9      Hungarian(int n, int m) : n(n), m(m), adj(n),
        ↳ match_v(m, -1), visited(m, false) {}
10
11     void addEdge(int u, int v) {
12         adj[u].push_back(v);
13     }
14
15     // DFS to find an augmenting path
16     bool dfs(int u) {
17         for (int v : adj[u]) {
18             if (visited[v]) continue;
19             visited[v] = true;
20
21             // If v is not matched, or its match can
22             ↳ find another choice
23             if (match_v[v] == -1 || dfs(match_v[v])) {
24                 match_v[v] = u;
25                 return true;
26             }
27         }
28         return false;
29     }
30
31     // Returns the maximum matching size
32     int solve() {
33         int ans = 0;
34         std::fill(match_v.begin(), match_v.end(), -1);
35         for (int i = 0; i < n; ++i) {
36             std::fill(visited.begin(), visited.end(),
37                 ↳ false);
38             if (dfs(i)) ans++;
39         }
40         return ans;
41     };

```

9. Sequence Algorithms

Longest Increasing Subsequence (LIS)

Finds the length of the longest strictly increasing subsequence in $O(N \log N)$ time and $O(N)$ space.

```

1  #include <vector>
2  #include <algorithm>
3
4  // Returns the length of the LIS
5  int getLIS(const std::vector<int>& nums) {
6      if (nums.empty()) return 0;
7

```

```

8      std::vector<int> v;
9      for (int x : nums) {
10         auto it = std::lower_bound(v.begin(), v.end(),
11             ↳ x);
12         if (it == v.end()) {
13             v.push_back(x); // x is larger than all
14             ↳ elements, extend the sequence
15         } else {
16             *it = x; // Replace the first element >= x
17             ↳ to maintain a smaller tail
18         }
19     }
20     return v.size();
21 }

```

10. Trie (Prefix Tree)

Description: A Trie (Prefix Tree) efficiently stores a collection of strings by sharing common prefixes. It supports insertion, exact string lookup, and prefix queries. **Time Complexity:** $O(L)$ for insert, search, and startsWith, L : The length of the string. **Space Complexity:** $O(\Sigma L)$, ΣL : The total number of inserted characters.

```

1  #include <vector>
2  #include <string>
3
4  using namespace std;
5
6  struct Trie {
7      // Each Trie node stores:
8      // nxt[i] : the index of the child corresponding to
9      ↳ character ('a' + i)
10     // isEnd : true if a word ends at this node
11     struct Node {
12         int nxt[26];
13         bool isEnd;
14
15         Node() {
16             fill(nxt, nxt + 26, -1);
17             isEnd = false;
18         }
19     };
20
21     vector<Node> trie;
22
23     Trie() {
24         trie.emplace_back(); // Node 0 is the root
25     }
26
27     // Inserts a string into the Trie.
28     void insert(const string& word) {
29         int current = 0;
30
31         for (char ch : word) {
32             int index = ch - 'a';
33
34             // Create a new node if the path does not
35             ↳ exist
36             if (trie[current].nxt[index] == -1) {
37                 trie[current].nxt[index] = trie.size();
38                 trie.emplace_back();
39             }
40         }
41     }

```



```

38         current = trie[current].nxt[index];
39     }
40
41     trie[current].isEnd = true;
42 }
43
44 // Returns true if the exact word exists in the
45 // Trie.
46 bool search(const string& word) {
47     int current = 0;
48
49     for (char ch : word) {
50         int index = ch - 'a';
51
52         if (trie[current].nxt[index] == -1)
53             return false;
54
55         current = trie[current].nxt[index];
56     }
57
58     return trie[current].isEnd;
59 }
60
61 // Returns true if there exists any word with the
62 // given prefix.
63 bool startsWith(const string& prefix) {
64     int current = 0;
65
66     for (char ch : prefix) {
67         int index = ch - 'a';
68
69         if (trie[current].nxt[index] == -1)
70             return false;
71
72         current = trie[current].nxt[index];
73     }
74
75     return true;
76 }

```

11. Aho-Corasick Automaton (AC Automaton)

Description: An extension of Trie that supports efficient **multiple pattern matching** by adding failure links (similar to KMP). It can find all occurrences of multiple patterns in a text in linear time.

Use AC Automaton when:

- Matching **multiple patterns** in a single text
- Sensitive word filtering
- Dictionary matching
- DNA sequence matching
- String DP (Automaton DP)

If there is only **one pattern**, KMP is usually sufficient.

Algorithm Steps

1 Patterns -> Build Trie -> Build Failure Links (BFS) ->
 ↳ Scan Text

Failure links allow the automaton to continue matching from the **longest valid suffix** instead of restarting from the root.

```

1 struct AhoCorasick {
2     static constexpr int SIGMA = 26;
3
4     struct Node {
5         int nxt[SIGMA];
6         int fail;
7         vector<int> out;
8
9         Node() {
10             memset(nxt, 0, sizeof(nxt));
11             fail = 0;
12         }
13     };
14
15     vector<Node> trie;
16
17     AhoCorasick() {
18         trie.emplace_back(); // root
19     }
20
21     void insert(const string &s, int id) {
22         int cur = 0;
23         for (char c : s) {
24             int x = c - 'a';
25             if (!trie[cur].nxt[x]) {
26                 trie[cur].nxt[x] = trie.size();
27                 trie.emplace_back();
28             }
29             cur = trie[cur].nxt[x];
30         }
31         trie[cur].out.push_back(id);
32     }
33
34     void build() {
35         queue<int> q;
36
37         for (int c = 0; c < SIGMA; c++) {
38             int v = trie[0].nxt[c];
39             if (v) q.push(v);
40         }
41
42         while (!q.empty()) {
43             int u = q.front();
44             q.pop();
45
46             for (int c = 0; c < SIGMA; c++) {
47                 int &v = trie[u].nxt[c];
48
49                 if (v) {
50                     trie[v].fail =
51                     ↳ trie[trie[u].fail].nxt[c];
52                     trie[v].out.insert(
53                         trie[v].out.end(),
54                         trie[trie[v].fail].out.begin(),
55                         trie[trie[v].fail].out.end()
56                     );
57                 }
58             }
59             q.push(v);
60         }
61     }
62 }

```

```

56         q.push(v);
57     } else {
58         v = trie[trie[u].fail].nxt[c];
59     }
60 }
61 }
62 }
63
64 vector<int> query(const string &text) {
65     vector<int> res;
66     int cur = 0;
67
68     for (char c : text) {
69         cur = trie[cur].nxt[c - 'a'];
70         for (int id : trie[cur].out)
71             res.push_back(id);
72     }
73
74     return res;
75 }
76 };

```

Procedure

1. Insert all patterns into the Trie.
2. Build failure links using BFS.
3. Scan the text once.
4. Report every matched pattern.

Notes

- Root index is usually 0.
- Failure links are built using **BFS**.
- Missing transitions are completed during **build()**, so matching is $O(|\text{Text}|)$.
- **out** stores all pattern IDs ending at the current node.
- Frequently combined with **DP** to solve forbidden-string and string-counting problems.

12. Expected Value DP

Idea: Define the expectation of each state and write the expectation equation directly.

General Formula

$$E(s) = \sum P_i (C_i + E(next_i))$$

where

- P_i : probability of transition
- C_i : immediate cost
- $E(next_i)$: future expectation

If Self-loop Exists

Suppose

$$E(s) = A + pE(s)$$

Move the self-loop term to the left:

$$E(s) = \frac{A}{1-p}$$

This is the most common transformation in Expected Value DP.

Steps

1. Define the DP state.
2. Enumerate all possible transitions.
3. Write the expectation equation directly.
4. Move self-loop terms to the left if necessary.
5. Solve the equation.
6. Use memoization or DP.

Common State Types

- $dp[i]$
- $dp[mask]$
- $dp[x][y]$
- $dp[a][b][c]$